Chapter 1: Introduction

1.1 Background

Smartphones are always with us, which makes them the natural tool for asking for help. However, during an emergency, calling, messaging, sharing location and giving medical information are done through separate apps and several manual steps. Panic, injury or an unfamiliar place make these steps slow and error-prone.

1.2 Motivation

The project was chosen because a well-designed mobile app can reduce the time between "something is wrong" and "help is informed" to a few seconds. It also covers many of the course topics in a single, meaningful application: UI design, navigation, forms, local database, state management, device sensors and services.

1.3 Scope

RescuSphere v1.0.0 is an Android-first application for individual users in India. It works without an account or internet connection for its core features. It prepares and opens an SMS to saved contacts; sending is confirmed by the user. Cloud backup, login and background sending are outside the current scope (see Future Scope).

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

Chapter 2: Problem Statement

Background

In an emergency such as a medical crisis, a fire or a security threat, the first few minutes decide how well the situation can be handled. The person in trouble is usually frightened and may be injured. They often cannot type, search for numbers or explain where they are. Smartphones are always at hand, but they are not designed for this moment.

The Problem in Detail

Today, getting help takes several separate steps spread across different apps:

1. **Remembering the right helpline.** India has several emergency numbers (112 National SOS, 108 Ambulance, 100 Police, 101 Fire), and a stressed person may not remember which one to call.
2. **Explaining the location.** In an unfamiliar area, on a road or in a large building, people often cannot describe exactly where they are.
3. **Informing family and friends.** This is done manually through calls or messages, which takes time and may not happen at all if the person cannot type.
4. **Communicating medical information.** Blood group, allergies and medical conditions are known only to the patient. If the patient is unconscious or panicking, responders and helpers do not have this information.
5. **Describing the emergency.** There is no simple way to say what kind of emergency it is, how serious it is and what help is needed (ambulance, police or fire).
6. **Avoiding false alarms.** A one-touch alert can be triggered by accident, so it needs a safeguard.
7. **Keeping a record.** After the event there is usually no record of when and where an alert was raised, or whether it was resolved.

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

Chapter 3: Literature and Existing System Study

3.1 The Problem in Today's Emergency Process

In an emergency a person has to remember the correct emergency number (112, 108, 100 or 101), explain where they are, inform family or friends separately, and tell responders about allergies or medical conditions. These steps are spread over the dialer, messaging apps, maps and notes, so valuable time is lost exactly when it is least affordable.

3.2 Existing Solutions

Existing Solution	What It Does	Limitation
Dialling 112 / 108 / 100 / 101 directly	Connects to the national emergency services	Needs the caller to speak, explain the location and remember the right number; does not inform family or share medical details
Built-in phone Medical ID / Emergency Information (Android and iOS)	Stores medical details and emergency contacts for viewing from the lock screen	Passive only: it does not send alerts, share GPS location or keep any incident history
Government emergency apps (for example, the 112 India app)	Official channel for raising emergency requests	Focused on dispatch to services; not designed around a personal circle of contacts, a stored medical profile or a personal incident history
Personal safety apps (for example, bSafe, Google Personal Safety)	Share live location and send alerts to trusted contacts	Usually need an account, internet connection and cloud storage of personal data; limited integration with Indian helpline numbers

3.3 Limitations of Existing Solutions

1. **Fragmented experience:** calling, messaging, location sharing and medical information are in different apps.

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

2. **Too many steps under stress:** most solutions require typing, searching or navigating several screens.
3. **No guided reporting:** the person must decide what to say; there is no structured way to describe the type and severity of the emergency.
4. **Dependence on accounts and internet:** many safety apps require registration and online storage, which is a problem with poor connectivity and a privacy concern for medical data.
5. **No personal incident history:** past emergencies cannot be reviewed, marked as resolved or deleted by the user.

3.4 Target Users

User Group	Why They Need the App
Students and hostel residents	Often live away from family and need a fast way to alert trusted contacts
Women and people travelling alone	Need a quick, one-tap way to share live location with trusted people
Elderly people and people with medical conditions	Need their blood group, allergies and conditions available to responders
Families with children or elderly members	Want family members to be alerted instantly with a location link
General public and travellers	Need one place with all Indian emergency numbers and a guided reporting flow

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

Chapter 4: Proposed System

4.1 Proposed Solution

RescuSphere is a single mobile application that brings every emergency action together on one screen:

- A large **SOS button** with a 3-second countdown (and a cancel option, to avoid false alarms) that captures the GPS location and prepares an alert message for the saved ICE (In Case of Emergency) contacts.
- A **Smart Triage** flow in which the user selects the emergency type, severity and quick details in three taps.
- A **Medical ID** that stores the user's name, blood group, allergies and medical conditions, and automatically includes them in the alert message.
- **One-tap calling** to 112 (National SOS), 108 (Ambulance), 100 (Police) and 101 (Fire).
- An **Incident Log** that records every alert with time, type, status and location, so it can be reviewed, marked resolved or deleted.
- A **strobe visual alarm** and **quick location sharing** for situations where the user needs to attract attention or simply share where they are.
- All data is stored **locally on the device** using SQLite, so personal medical information stays private and the app does not need an account or sign-up.

4.2 Proposed Features

Module	Features
SOS Alert	Pulsing SOS button; 3-second countdown with cancel; vibration feedback; automatic GPS capture; pre-filled alert message (location link and medical details) for all ICE contacts; incident saved automatically
Smart Triage	Step 1: choose emergency type (Medical, Fire, Security); Step 2: choose severity (Low, Medium, High); Step 3: choose quick info (Need Ambulance, Need Police, Need Firetruck, Trapped, Bleeding, Unconscious) or skip; alert sent with location
Medical ID	Create, edit and reset profile; name; blood group chosen from 9 options; allergies and medical conditions with quick-select chips

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

Module	Features
	and free text
ICE Contacts	Add contact (name, relationship, phone number); list contacts; call a contact; delete a contact
Emergency Hotlines	One-tap dial for 112, 108, 100 and, on the Medical ID screen, 101
Incident Logs	Chronological list; filter by All / Active / Resolved; mark as resolved or reopen; delete one log; clear all logs (with confirmation); open incident location in maps
Quick Share Location	Fetches live GPS location, copies the message to the clipboard and prepares an SMS to ICE contacts
Strobe Visual Alarm	Flashing red screen to attract attention; toggled on and off
Navigation and UI	Navigation drawer, dark theme with emergency colour scheme, glass-style cards, status badges on the home screen

The home screen of the application is shown in Figure 11.1.

4.3 Scope and Limitations of the Current Version

- The alert SMS is **prepared and opened in the phone's SMS application**; the user confirms by pressing Send.
- Data is stored **only on the device**; there is no cloud backup and no user login in this version.
- The app depends on **GPS and phone network** availability for location and messages.
- Hotline numbers (112, 108, 100, 101) are specific to **India**.

Chapter 5: Functional and Non-Functional Requirements

This chapter lists what RescuSphere must do (functional requirements), how well it must do it (non-functional requirements), who uses it (user roles), how they use it (use cases and user stories) and what is needed to build and run it (system requirements). The requirements were derived from the problem statement and checked against the implemented application (version 1.0.0).

Priority key: High = essential for an emergency; Medium = important; Low = useful extra. **Status key:** Implemented = working in the current build; Partial = works with a limitation, explained in the notes below the table.

5.1 User Roles and Actors

Actor	Type	Description	Interaction with the App
User (Person in need)	Primary	The owner of the phone who sets up the Medical ID and ICE contacts and raises alerts	Uses every screen of the app
ICE Contact (family / friend)	Secondary (external)	A trusted person saved by the user	Receives the alert SMS with location link and medical details; can be called by the user
Emergency Services (112, 108, 100, 101)	Secondary (external)	National emergency helplines	Called from the phone dialer when the user taps a hotline button
Device Services	System	GPS, vibration motor, SMS application, phone dialer, maps application	Used by the app to get location, give feedback, send the message, place calls and show maps

The app has **a single user role**. There is no login, administrator or multi-user feature in this version; the data belongs to the person using the device.

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

5.2 Functional Requirements

ID	Requirement	Module	Priority	Status
FR-01	The system shall provide a large SOS button on the home screen that starts a 3-second countdown when tapped.	SOS Alert	High	Implemented
FR-02	The user shall be able to cancel the SOS by tapping the button again during the countdown.	SOS Alert	High	Implemented
FR-03	The system shall give vibration feedback when the SOS is pressed, on each countdown step and when the alert is activated.	SOS Alert	Medium	Implemented
FR-04	On activation, the system shall obtain the current GPS location with high accuracy (8-second limit) and fall back to the last known location if this fails.	Location	High	Implemented

ID	Requirement	Module	Priority	Status
FR-05	The system shall check that location services are on and that permission is granted, ask for permission when needed, and show a message (and open settings) if it is denied.	Location	High	Implemented
FR-06	The system shall build an alert message containing the emergency type, severity and details (where given), a Google Maps link, the coordinates, and the user's Medical ID details.	Alerts	High	Implemented
FR-07	The system shall open the phone's SMS application with the alert message pre-filled and addressed to all saved ICE contacts.	Alerts	High	Partial
FR-08	The system shall warn the user if	Alerts	Medium	Implemented

ID	Requirement	Module	Priority	Status
	no ICE contact is saved when an alert is triggered.			
FR-09	The system shall save every alert in the incident log with time, type, status (Active), summary and coordinates.	Incident Logs	High	Implemented
FR-10	The system shall provide a three-step Smart Triage: choose type (Medical, Fire, Security), choose severity (Low, Medium, High), choose quick info (six options) or skip.	Smart Triage	High	Implemented
FR-11	The user shall be able to create, edit and reset a Medical ID: name, blood group (9 options), allergies and medical conditions, with quick-select chips and free text.	Medical ID	High	Implemented

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

ID	Requirement	Module	Priority	Status
FR-12	The user shall be able to add an ICE contact (name, relationship, phone number), view the list, call a contact and delete a contact. The phone number is mandatory.	ICE Contacts	High	Implemented
FR-13	The system shall provide one-tap dialling for 112 (National SOS), 108 (Ambulance), 100 (Police) and, on the Medical ID screen, 101 (Fire).	Hotlines	High	Implemented
FR-14	The system shall list incident logs newest first and let the user filter them as All, Active or Resolved, with a count for each.	Incident Logs	Medium	Implemented
FR-15	The user shall be able to mark an incident as Resolved or reopen it as Active.	Incident Logs	Medium	Implemented

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

ID	Requirement	Module	Priority	Status
FR-16	The user shall be able to delete one incident log, or clear all logs, after a confirmation dialog.	Incident Logs	Medium	Implemented
FR-17	The user shall be able to open the location of an incident in a maps application (when coordinates are available).	Incident Logs	Low	Implemented
FR-18	The system shall provide Quick Share Location: get GPS location, copy the message to the clipboard and prepare an SMS to the ICE contacts.	Location Sharing	Medium	Implemented
FR-19	The system shall provide a Strobe Visual Alarm that flashes the screen red and can be switched off.	Alarm	Low	Implemented
FR-20	The home screen shall show the status	Navigation	Medium	Implemented

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

ID	Requirement	Module	Priority	Status
	of the Medical ID and the number of ICE contacts, and a navigation drawer shall give access to Incident Logs (with count), Medical ID and ICE contacts, Smart Triage and the Strobe alarm.			
FR-21	The system shall store the Medical ID, ICE contacts and incident logs in a local SQLite database so that the data remains after the app is closed.	Data Storage	High	Implemented

Note on FR-07: the alert message is prepared and the SMS application opens with the recipients and text already filled in, but the user has to press Send. The app does not send the SMS silently in the background.

5.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	The SOS button shall be the most prominent element on the home screen and the main emergency actions shall be reachable in a few taps; triage shall take no more

ID	Category	Requirement
		than three selections.
NFR-02	Usability	The interface shall use a dark theme with high-contrast red and light text so that it can be read quickly under stress.
NFR-03	Safety	The countdown and cancel option shall reduce accidental alerts. Destructive actions (delete log, clear logs, reset profile) shall ask for confirmation.
NFR-04	Performance	Screens shall respond without noticeable delay; the location request shall not block the app for more than 8 seconds before falling back to the last known position.
NFR-05	Reliability	If GPS is unavailable, the app shall still record the incident and tell the user that the location could not be acquired.
NFR-06	Availability	All core features (Medical ID, contacts, logs, triage, hotlines) shall work without an internet connection. Sending the SMS needs mobile network coverage; opening a map link needs internet.
NFR-07	Privacy and Security	Personal and medical data shall be stored only on the user's device. No account is required, and the source code shall not contain

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

ID	Category	Requirement
		passwords or API keys.
NFR-08	Portability	The app shall target Android as the primary platform; the Flutter code base also allows building for other platforms.
NFR-09	Compatibility	The layout shall adapt to different phone screen sizes.
NFR-10	Maintainability	The code shall be organised in separate folders for models, database, providers, screens, services, widgets and theme, using meaningful names.
NFR-11	Error Handling	The app shall show clear messages for denied permissions, disabled GPS, missing contacts and failure to open the SMS or dialer application.

5.4 Use Cases

Use Case Summary

ID	Use Case	Primary Actor	Related Requirements
UC-01	Trigger SOS Alert	User	FR-01 to FR-09
UC-02	Report Emergency using Smart Triage	User	FR-04 to FR-10
UC-03	Manage Medical ID	User	FR-11, FR-21
UC-04	Manage ICE Contacts	User	FR-12, FR-21
UC-05	Call Emergency Helpline	User	FR-13

ID	Use Case	Primary Actor	Related Requirements
UC-06	View and Filter Incident Logs	User	FR-14
UC-07	Update or Delete Incident Log	User	FR-15, FR-16
UC-08	View Incident Location on Map	User	FR-17
UC-09	Quick Share Live Location	User	FR-18
UC-10	Use Strobe Visual Alarm	User	FR-19

UC-01: Trigger SOS Alert

Item	Description
Actor	User (secondary: ICE Contact, Device Services)
Precondition	App is open on the home screen. For a complete alert, at least one ICE contact and a Medical ID are saved.
Main Flow	1. User taps the SOS button. 2. The countdown starts from 3 and the phone vibrates. 3. The countdown ends and the app vibrates in a pattern. 4. The app gets the GPS location. 5. The app saves an Active incident log. 6. The app builds the alert message with the location link and Medical ID. 7. The SMS application opens with the message addressed to all ICE contacts. 8. User presses Send. 9. The app shows a confirmation message.
Alternate Flows	A1: User taps the button during the countdown, so the SOS is cancelled and a message is shown. A2: Location services are off, so the app asks the user to enable them.

Item	Description
	A3: Permission is denied, so the app shows a message and the alert is logged without coordinates. A4: No ICE contacts exist, so the app shows a warning to add contacts. A5: The SMS application cannot be opened, so an error message is shown.
Postcondition	An incident with status Active is stored in the database.

UC-02: Report Emergency using Smart Triage

Item	Description
Actor	User
Precondition	App is open.
Main Flow	1. User opens Smart Triage from the home screen or the drawer. 2. User selects the type: Medical, Fire or Security. 3. User selects the severity: Low, Medium or High. 4. User selects a quick-info option (for example Bleeding or Trapped) or taps Skip / Send Now. 5. The app gets the location, saves the incident and prepares the SMS to ICE contacts. 6. The app shows a confirmation and returns to the home screen.
Alternate Flows	Same location, permission and contact alternatives as UC-01.
Postcondition	An incident with the selected type, severity and info is stored with status Active.

UC-03 and UC-04: Manage Medical ID and ICE Contacts

Item	Description
Actor	User
Main Flow (Medical ID)	The user opens Medical ID and ICE, taps Create or Edit, enters the name, picks a blood

Item	Description
	group, adds allergies and conditions (chips or text) and saves. The user may also reset the profile after confirming.
Main Flow (ICE Contacts)	The user taps Add Contact, enters name, relationship and phone number, and taps Add. The contact appears in the list. The user can call or delete a contact from the list.
Alternate Flow	If the phone number is empty, the contact is not added.
Postcondition	The data is saved in the local database and shown on the home screen status badges.

The use-case diagram is given in Figure 8.2.

5.5 User Stories

ID	User Story	Acceptance Criteria
US-01	As a user in danger, I want to press one big button so that my contacts are alerted without typing.	Tapping SOS starts a 3-second countdown and then opens a pre-filled SMS to all ICE contacts.
US-02	As a user, I want to cancel an accidental SOS so that my contacts are not alarmed.	Tapping the button during the countdown stops the SOS and shows a cancelled message.
US-03	As a user, I want my live location in the alert so that people know where to find me.	The message contains a Google Maps link and coordinates when GPS is available.
US-04	As a user with allergies or a medical condition, I want my blood group and conditions sent with the alert so that responders can act correctly.	The message includes name, blood group, allergies and conditions saved in the Medical ID.
US-05	As a user, I want to describe	Triage collects type, severity

ID	User Story	Acceptance Criteria
	the emergency in a few taps so that helpers know what kind of help is needed.	and quick info in three steps, and these appear in the alert and the log.
US-06	As a user, I want to save trusted contacts so that the right people are always alerted.	A contact with a phone number can be added, called and deleted.
US-07	As a user, I want to call 112, 108, 100 or 101 with one tap so that I do not have to remember or type numbers.	Tapping a hotline opens the dialer with the number filled in.
US-08	As a user, I want to see my past incidents so that I can review what happened.	Logs are listed newest first with type, status, time and summary, and can be filtered.
US-09	As a user, I want to mark an incident as resolved or delete it so that my history stays accurate.	Status can be toggled and logs can be deleted after confirmation.
US-10	As a user, I want to share my location quickly without raising a full alert so that a friend can find me.	Quick Share copies the location message and opens an SMS to ICE contacts.
US-11	As a user, I want a flashing alarm so that I can attract attention.	The strobe flashes the screen red until switched off.
US-12	As a privacy-conscious user, I want my medical data to stay on my phone so that it is not exposed online.	No sign-up is needed and data is stored in a local database.

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

5.6 System Requirements

5.6.1 End-User Device Requirements

Device	Android smartphone
Operating System	Android version supported by the app's Flutter build (default minimum set by the Flutter SDK)
Hardware	GPS receiver, vibration motor, SIM card with SMS capability
Storage	Small (the database holds only text records)
Network	Mobile network for SMS and calls; internet only for opening map links

5.6.2 Development Environment

Item	Requirement
Framework	Flutter 3.44 or later, Dart 3.12 or later (as resolved in `pubspec.lock`)
IDE	Android Studio or Visual Studio Code with the Flutter and Dart plugins
Android Build	Compile SDK 36, Java 17 compatibility, Gradle (Kotlin DSL)
Database	SQLite through `sqflite` (and `sqflite_common_ffi` for Windows and Linux testing)
Version Control	Git and GitHub
Testing Devices	Android emulator and at least one physical Android phone (GPS, calling and SMS cannot be fully tested on an emulator)

5.6.3 Android Permissions

Permission	Why It Is Needed
`ACCESS_FINE_LOCATION`,	To read the GPS position for the alert

Permission	Why It Is Needed
`ACCESS_COARSE_LOCATION`	
`VIBRATE`	Vibration feedback during SOS
`INTERNET`	Opening the maps link
`SEND_SMS`, `CALL_PHONE`	Declared in the manifest. The current version uses the SMS application and the dialer, so the user confirms sending and calling.

5.7 Assumptions and Constraints

Assumptions

1. The user has granted location permission and has switched GPS on when an alert is raised.
2. The ICE contact phone numbers entered by the user are correct.
3. The phone has network coverage when an SMS or call is made.

Constraints

1. The app depends on the phone's SMS and dialer applications, so the final Send or Call press is made by the user.
2. Emergency numbers (112, 108, 100, 101) are specific to India.
3. There is no cloud backup in this version; uninstalling the app removes the data.
4. First-launch data includes a sample profile and sample incident logs for demonstration; the user can edit or reset them.

Chapter 6: Technology Stack

Component	Technology	Version	Purpose and Justification
Framework	Flutter (Dart)	Flutter 3.44 or later, Dart 3.12 or later	One code base, fast and smooth UI, rich widgets; suits the animated SOS button
State management	`provider`	^6.1.2	Simple, recommended way to share data between screens and refresh them on change
Local database	`sqflite`	^2.3.3	On-device SQLite storage; works offline and keeps medical data private
Desktop database support	`sqflite_common_ffi`	^2.3.0	Lets the same database code run on Windows and Linux for testing
File paths	`path`	^1.9.0	Builds the database file path
Location	`geolocator`	^14.1.1	GPS location, permission checks, last-known-position fallback
Calls, SMS and maps	`url_launcher`	^6.3.2	Opens the dialer, SMS application and maps link
Vibration	`vibration`	^3.2.1	Physical feedback during countdown and alert
Audio	`audioplayers`	^6.8.1	Declared for alarm sound (not yet used,

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

Component	Technology	Version	Purpose and Justification
			see the bug log)
Date formatting	`intl`	^0.19.0	Readable date and time in the incident logs
Icons	`cupertino_icons`	^1.0.6	Standard icon set
Tools	Android Studio or VS Code, Git, GitHub	–	Development and version control

App version: **1.0.0+1**. Android build uses compile SDK 36 and Java 17 compatibility.

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

Chapter 7: System Architecture and Design

7.1 Design Approach

RescuSphere is designed as a small layered Flutter application in which every screen reads its data from one shared state object and every state change is saved to a local SQLite database. This keeps the screens simple and keeps the emergency logic in one place.

Layer	Folder	Responsibility
Presentation	`screens/`, `widgets/`, `theme/`	The four screens (Home, Smart Triage, Medical ID and ICE, Incident Logs), reusable widgets (SOS button, glass card) and the dark emergency theme
State Management	`providers/app_provider.dart`	`AppProvider` (Provider package) holds the Medical profile, ICE contacts and incident logs, and notifies the screens when they change
Services	`services/emergency_service.dart`	`EmergencyService` handles location, vibration, calling, SMS, maps link, clipboard and builds the alert message
Data	`database/database_helper.dart`, `models/`	`DatabaseHelper` creates and queries the SQLite database; `MedicalProfile`, `IceContact` and `IncidentLog` are the data models

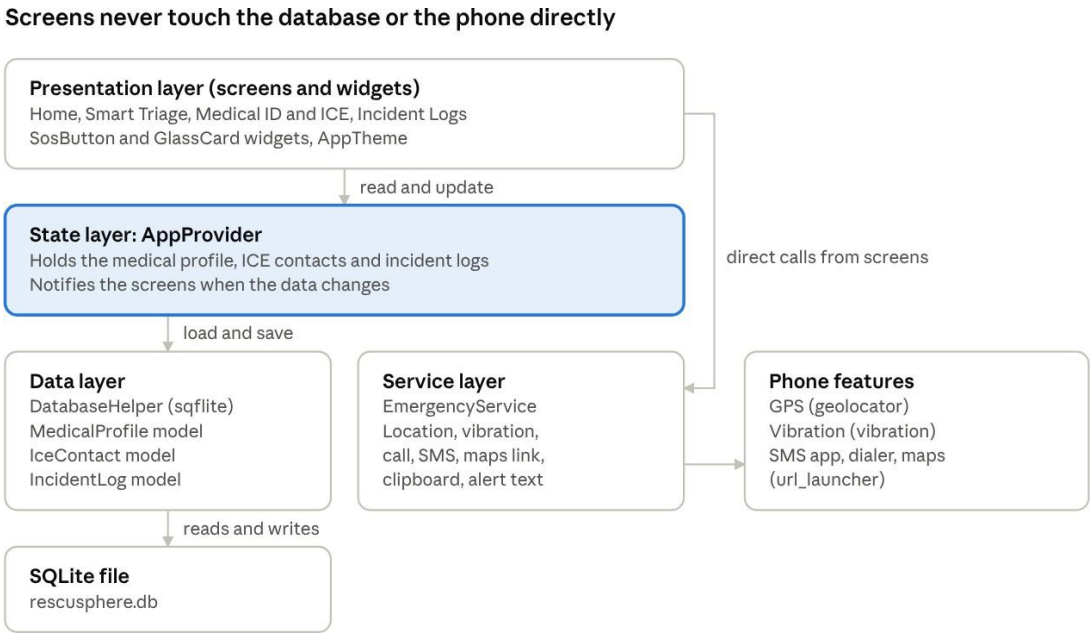
Design principles followed

1. **Speed under stress:** the SOS button is the largest element and reachable without any menu.
2. **Prevent mistakes:** a 3-second countdown with cancel, and confirmation dialogs for destructive actions.

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

3. **Privacy first:** all data stays in the on-device database; no sign-up and no cloud.
4. **Separation of concerns:** screens never talk to the database directly; they go through `AppProvider`.
5. **Consistent look:** one theme, red for emergency actions, blue for informational actions, dark background for readability.

7.2 System Architecture



layered architecture · 4 layers, database and phone features

Figure 7.1 – System architecture of RescuSphere. The screens read and update the shared `AppProvider`, which loads and saves through `DatabaseHelper` into the SQLite file. Actions that touch the phone (calling, SMS, location, vibration, maps) go straight from the screens to `EmergencyService`.

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

7.3 Use-Case Diagram

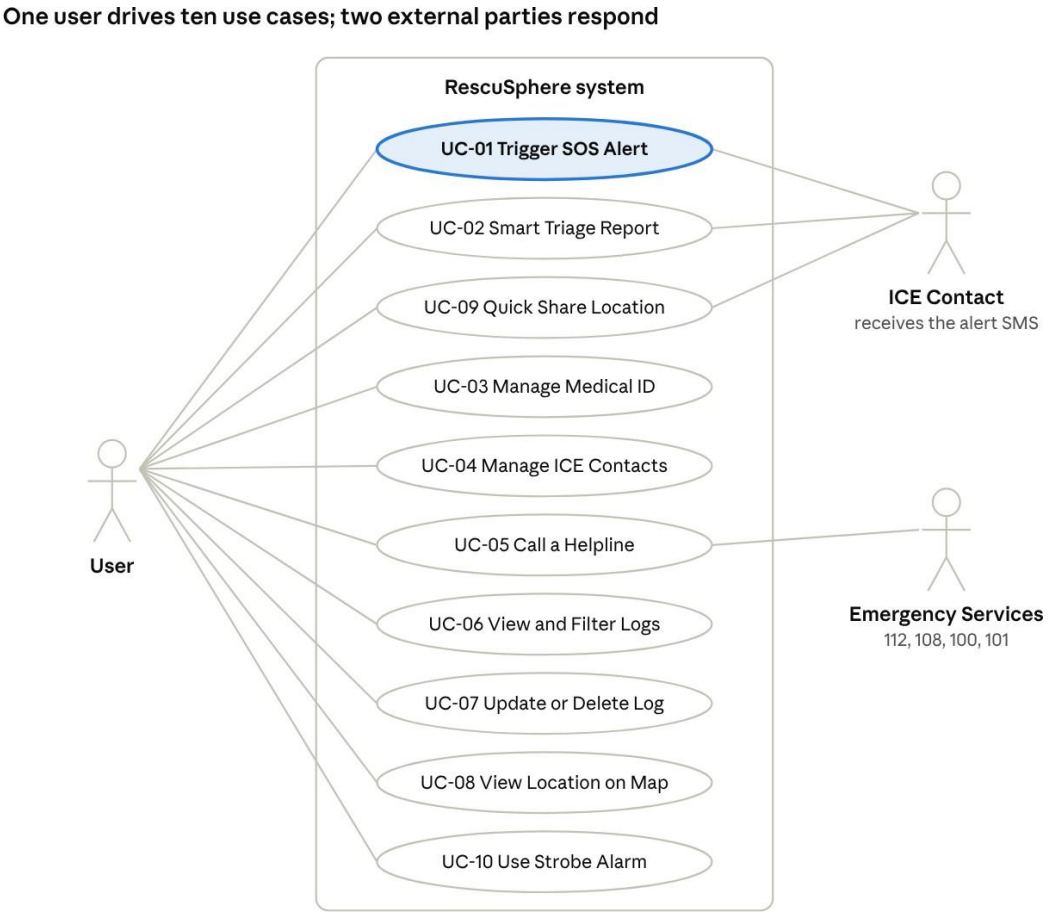
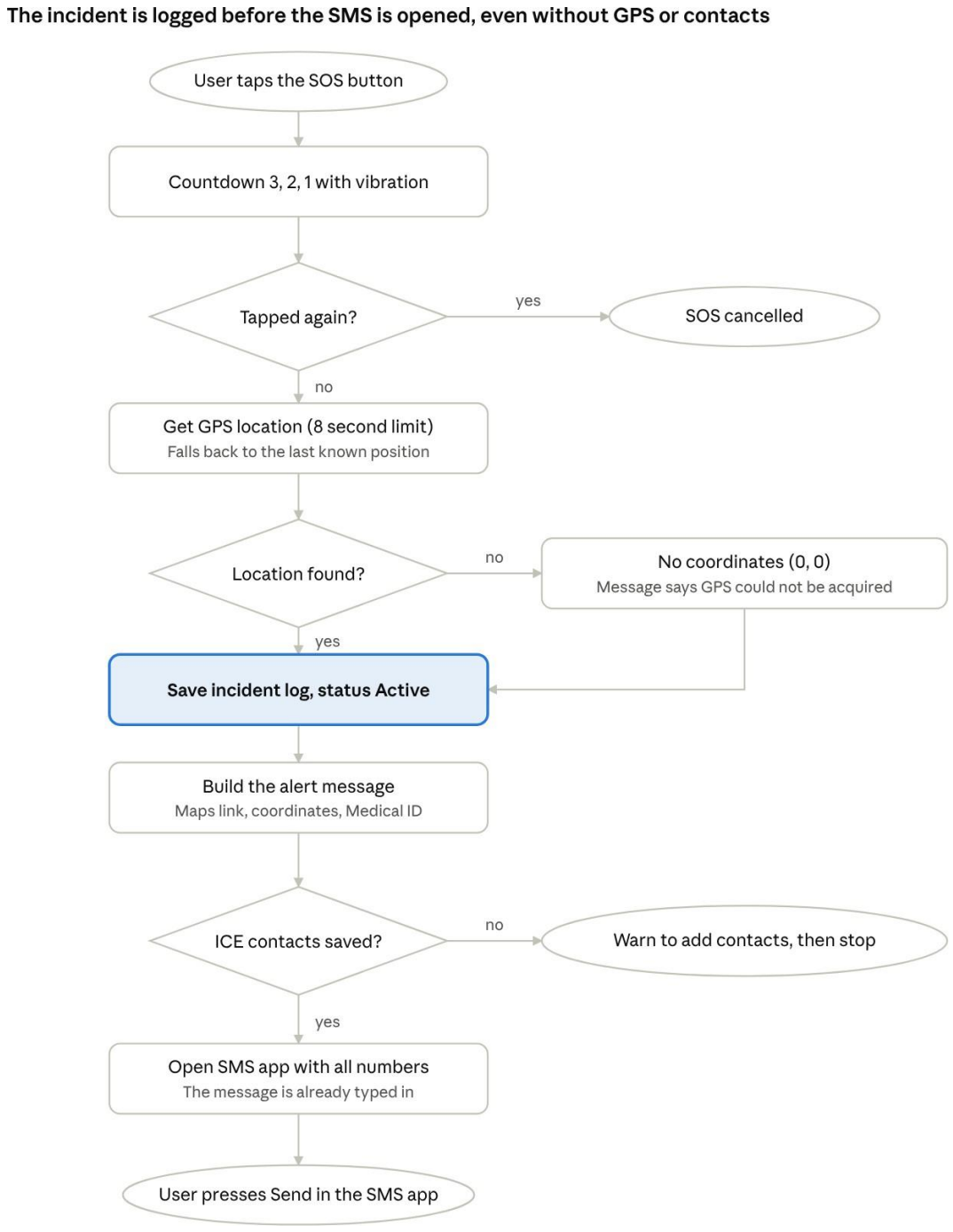


Figure 7.2 – Use-case diagram of RescuSphere. The user takes part in all ten use cases (described in Chapter 5). ICE contacts receive the SMS produced by UC-01, UC-02 and UC-09, and the emergency services are reached through the phone dialer in UC-05. The phone's GPS, vibration, SMS application and maps application are used behind the scenes and are not drawn as actors.

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

7.4 Activity Diagram – SOS Alert



activity diagram · SOS alert, 3 decisions

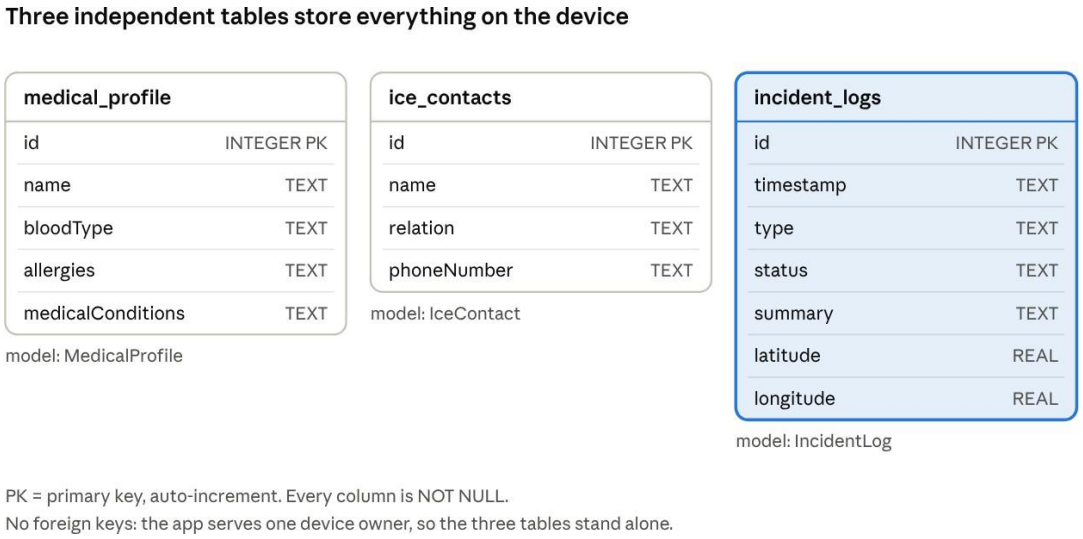
Figure 7.3 – Activity (flow) diagram of the SOS alert (UC-01). The user can cancel during the countdown. After that the app always saves an Active incident log, whether or not GPS coordinates were found. If no ICE contact is saved the user is warned and the flow stops; otherwise the SMS

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

application opens with the message ready and the user presses Send. Smart Triage (UC-02) follows the same flow after the type, severity and quick info are chosen.

7.5 Database Design

The app uses one SQLite database file, `rescusphere.db` (schema version 1), created on first launch by `DatabaseHelper`. The first launch also inserts a sample medical profile, a sample ICE contact and sample incident logs for demonstration; the user can edit or delete them.



ER diagram · 3 tables, 16 columns

Figure 7.4 – ER diagram of the RescuSphere database. Each table maps to one model class. The highlighted `incident_logs` table is the one that grows with every alert.

Data Dictionary

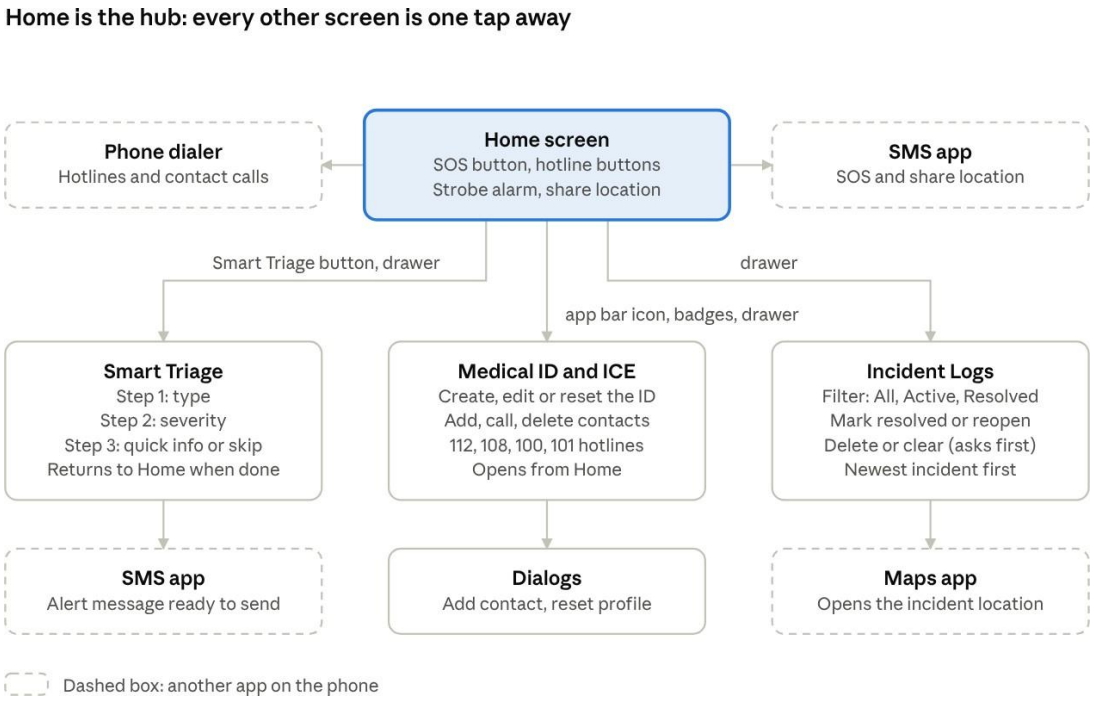
Table	Column	Data Type	Constraint	Description
medical_profile	id	INTEGER	Primary key, auto-increment	Unique profile id
medical_profile	name	TEXT	NOT NULL	Full name of the user
medical_profile	bloodType	TEXT	NOT NULL	Blood group, for example O+
medical_profile	allergies	TEXT	NOT NULL	Allergies as comma-separated text

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

Table	Column	Data Type	Constraint	Description
medical_profile	medicalConditions	TEXT	NOT NULL	Medical conditions as comma-separated text
ice_contacts	id	INTEGER	Primary key, auto-increment	Unique contact id
ice_contacts	name	TEXT	NOT NULL	Contact's name
ice_contacts	relation	TEXT	NOT NULL	Relationship, for example Mother or Friend
ice_contacts	phoneNumber	TEXT	NOT NULL	Number used for calls and for the alert SMS
incident_logs	id	INTEGER	Primary key, auto-increment	Unique incident id
incident_logs	timestamp	TEXT	NOT NULL	Date and time in ISO 8601 text format
incident_logs	type	TEXT	NOT NULL	SOS, Medical, Fire or Security
incident_logs	status	TEXT	NOT NULL	Active or Resolved
incident_logs	summary	TEXT	NOT NULL	Short description of the incident
incident_logs	latitude	REAL	NOT NULL	Latitude of the alert (0.0 if GPS was unavailable)
incident_logs	longitude	REAL	NOT NULL	Longitude of the alert (0.0 if GPS was unavailable)

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

7.6 Navigation Flow



navigation flow · 3 screens off one hub

Figure 7.5 – Navigation flow of RescuSphere. The Home screen is the hub. It opens Smart Triage, Medical ID and ICE, and Incident Logs through push navigation, and each of these returns to Home with the Back button. Calls, SMS and maps are handed over to other apps on the phone. The navigation drawer on Home lists Incident Logs, Medical ID and ICE Contacts, Smart Triage and the Strobe alarm.

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

Chapter 8: Implementation

8.1 Key Implementation Details

1. SOS button and countdown. `SosButton` is a stateful widget with a pulsing animation. When pressed it sets the countdown to 3 and starts a one-second periodic `Timer`; it vibrates on the press and on each step. Pressing the button again while the timer is running cancels it. When the countdown ends, the button vibrates in a long pattern and calls `EmergencyService.triggerEmergency`. A flag prevents a second SOS from starting while one is already active.

2. Location. `EmergencyService.getCurrentLocation` first checks whether location services are enabled and whether permission is granted, requests it when needed, and shows messages when it is not available. It then requests the position with high accuracy and an 8-second limit and falls back to the last known position if that fails. If nothing is available the app continues without coordinates.

3. Emergency flow. `triggerEmergency` gets the location, saves an incident with status Active through `AppProvider`, builds the alert text (type, severity and extra information when given, a Google Maps link, coordinates and the Medical ID details) and then sends it to the contacts. If no contact is saved, the user is warned instead.

4. SMS and calls. The app uses `url_launcher` with the `sms:` and `tel:` schemes. This opens the phone's own SMS application with all contact numbers and the message already filled in, and opens the dialer with the number typed in. The user presses Send or Call, so no special SMS permission is needed.

5. State management. `AppProvider` extends `ChangeNotifier`. Screens read data from it and call its methods to add, update or delete records; after each change it saves to SQLite and calls `notifyListeners`, so the home badges, drawer counts and lists refresh automatically.

6. Database. `DatabaseHelper` is a singleton that opens `rescusphere.db`, creates `medical_profile`, `ice_contacts` and `incident_logs` on first launch and inserts a sample profile, a sample contact and two sample incident logs. All CRUD operations are available for the three tables.

7. Smart Triage. `TriageScreen` keeps the current step and the selected values in its state, moves through type, severity and quick information, and then calls the same emergency flow as the SOS button with the chosen values.

8. Medical ID and contacts. `MedicalIdScreen` shows the profile and contacts, a form with a blood group dropdown (9 options) and chips for common allergies and conditions that append to the text field, and an Add Contact dialog where the phone number is mandatory.

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

9. Incident logs. `IncidentLogsScreen` lists logs newest first, filters by status, toggles Active and Resolved, deletes one log or all logs after confirmation and opens the stored coordinates in a maps application.

10. Strobe alarm. The strobe is a full-screen overlay that alternates colours on a timer to attract attention, and can be switched off by the user.

8.3 Coding Practices Followed

- Meaningful file, class and method names, grouped in folders for models, database, providers, screens, services, widgets and theme.
- A single service class for all device-related actions, so the screens stay simple.
- A reusable widget (`GlassCard`) and a central theme (`AppTheme`) for a consistent look.
- Confirmation dialogs for destructive actions and `try/catch` around device actions such as vibration and launching other apps.
- Source code kept in Git and GitHub for backup and version history.

Chapter 9: Testing and Test Cases

RescuSphere was tested by running its main features on an Android phone and by running the unit tests with `flutter test`. The 16 test cases below cover every major feature, input validation, failure situations, persistence, screen sizes, privacy and the unit tests. Problems found are listed in the bug log in Section 9.5.

9.1 Testing Approach

Type of Testing	Covered by Test Cases
Functional testing (SOS, triage, Medical ID, contacts, hotlines, logs, sharing, strobe)	TC-01 to TC-07, TC-09 to TC-12
Input validation and unexpected inputs	TC-08
Database and persistence testing	TC-06, TC-09, TC-10, TC-13
Failure testing (GPS off, permission denied, no contacts, no network)	TC-03, TC-04, TC-13
UI, navigation and screen-size testing	TC-14
Unit testing	TC-15
Security and privacy (no login in this version)	TC-16

9.2 Test Environment

Test Device	Android Version	Flutter / Dart	Build
One Plus Nord CE 2 Lite	CPH2381_14.0.0.3000	3.44 or later / 3.12 or later	Debug and Release APK

9.3 Test Cases

ID	Test Case and Steps	Expected Result	Result
TC-01	**SOS countdown and cancel.** Tap the SOS button, then tap it again within 3 seconds.	Countdown stops, "SOS Cancelled" is shown and no incident is saved.	Pass

Name: Aarish Bhanwadia
Enrollment No.: 12302080601001

ID	Test Case and Steps	Expected Result	Result
TC-02	**SOS alert.** GPS on, two contacts saved. Tap SOS and wait for the countdown to end.	Active incident saved; SMS app opens with both numbers and a message with maps link, coordinates and Medical ID.	Pass
TC-03	**SOS without GPS or permission.** Turn GPS off (or deny permission) and trigger SOS.	App asks to enable location; incident is saved with coordinates 0, 0; message says GPS was not acquired; no crash.	Pass
TC-04	**SOS without contacts.** Delete all contacts and trigger SOS.	Warning to add contacts; incident still logged; SMS app does not open.	Pass
TC-05	**Smart Triage.** Choose Medical, then High, then Bleeding.	Log and SMS contain type, severity and info; app returns to Home.	Pass
TC-06	**Medical ID save.** Edit name, blood group and allergies, save, then restart the app.	Success message; profile and home badge show new data; data remains after restart.	Pass
TC-07	**ICE contacts and hotlines.** Add a contact, call it, delete it; tap 112, 108, 100 and 101.	Contact is added, the dialer opens with the correct number each time, and the contact is removed on delete.	Pass
TC-08	**Input validation.** Save Medical ID with an empty name; add	Empty phone is rejected. An empty name and invalid	Pass

Name: Nisarg Barot
Enrolment No.: 12302080601086

ID	Test Case and Steps	Expected Result	Result
	a contact with an empty phone; add a contact with phone text "abc".	phone text should be rejected; the current build accepts them (BUG-03).	
TC-09	**Incident log filters and status.** Tap All, Active and Resolved; mark an incident Resolved and reopen it.	Correct incidents are listed, status changes and the counts update.	Pass
TC-10	**Delete log and clear all.** Delete one log (cancel once, then confirm); then Clear All.	Cancel keeps the log; confirm removes it; Clear All empties the list.	Pass
TC-11	**Open location in maps.** Open the map for an incident with coordinates.	Maps app opens at the stored location.	Pass
TC-12	**Quick share and strobe.** Tap the share-location icon; start and stop the strobe.	Location message is copied and SMS opens; screen flashes and stops when switched off.	Pass
TC-13	**Offline use and persistence.** Switch on airplane mode and use the in-app screens; close and reopen the app.	Profile, contacts and logs work offline and are still present after reopening.	Pass
TC-14	**Screen sizes.** Run on a small phone; rotate to landscape; set the largest system font.	All screens fit, scroll instead of overflowing, and nothing crashes.	Pass

Name: Nisarg Barot
Enrolment No.: 12302080601086

ID	Test Case and Steps	Expected Result	Result
TC-15	**Unit tests.** Run `flutter test`.	The three model tests pass. The Medical Profile test is expected to fail (BUG-01).	Pass
TC-16	**Privacy and source scan.** Search the code for keys, passwords and personal data; observe network use.	No keys or passwords; the app makes no network calls of its own. A personal phone number is found in the code (BUG-06).	Pass

9.4 Bug Log

The bugs were found by reading the source code and the existing test file.

ID	Issue	Severity	Suggested Fix
BUG-01	The Medical Profile unit test checks the key `blood_type`, but the model stores `bloodType`, so the test fails.	Low	Change the key in the test to `bloodType`.
BUG-02	After an SOS the message says "Alerts sent!", but the app only opens the SMS application; the user still has to press Send.	Medium	Reword it to say the SMS is ready and must be sent by the user.
BUG-03	Weak validation: the Medical ID can be saved with an empty name, and a contact can have a blank name or any text as	Medium	Require a name and check the phone number format.

ID	Issue	Severity	Suggested Fix
	the phone number.		
BUG-04	The database starts with a sample profile and sample contact. An SOS before editing them goes to the sample contact.	Medium	Remove the sample contact or add a first-run setup screen.
BUG-05	The `audioplayers` package is declared but no sound is played, and the manifest declares `SEND_SMS` and `CALL_PHONE` although the SMS and dialer apps are used instead.	Low	Add the alarm sound or remove the unused code and permissions.
BUG-06	A real-looking personal phone number appears in the unit test and as a hint in the contact dialog, and the app is named "emergency_app" with the id `com.example.emergency_app`.	Low	Use a dummy number; set the app name to RescuSphere and a proper application id.

Known limitation: when GPS fails, the coordinates are stored as 0, 0 to mean "no location". This is acceptable for this project and is listed in the future scope (nullable coordinates).

Conclusion of testing: the core emergency flows are covered by the test cases above. The bugs found are small and do not stop the application from working; the most important ones are BUG-02 (wording), BUG-03 (validation) and BUG-04 (sample contact).

Chapter 10: Challenges and Solutions

The table lists the main technical challenges in building an emergency application and how the design handles them.

Challenge	Solution Used
Accidental alerts from an accidental press	A 3-second countdown with a cancel option, plus confirmation dialogs for destructive actions
GPS may be off, denied or slow	Checks for services and permission, asks the user, uses an 8-second limit and the last known position, and still records the incident without coordinates
Sending SMS directly needs sensitive permissions and is restricted on many devices	Opens the phone's SMS application with the message and numbers filled in, so the user confirms sending
Keeping all screens in sync with stored data	Provider state management with a single `AppProvider` that saves to SQLite and notifies listeners
Medical data is private	Local SQLite storage only, no account and no cloud
SQLite on desktop platforms for development	`sqflite_common_ffi` is initialised on Windows and Linux
Making the interface usable under stress	Large SOS button, high-contrast dark theme and a few taps for every important action

Name: Nisarg Barot
Enrollment No.: 12302080601086

Chapter 11: Conclusion

RescuSphere shows that the most important emergency actions can be combined in one simple mobile application: raising an alert, sharing the exact location, passing on medical information, calling the right helpline and keeping a record. The project applies the main topics of the Mobile Application Development course in a real-world application with a clear purpose, and its testing identified a small number of improvements, mainly input validation and wording, that are listed in the bug log. The work has strengthened my understanding of Flutter, state management, local databases, device services, software testing and technical documentation.

Chapter 12: Future Scope

1. Automatic background SMS sending after the user grants the required permission.
2. Continuous live-location sharing with trusted contacts.
3. User authentication and cloud backup of the Medical ID, so it survives a reinstall.
4. Nearby hospitals, police stations and fire stations using a maps and places service.
5. Alarm sound and shake or volume-button triggers for the SOS.
6. Stronger validation, a first-run setup screen and removal of sample data.
7. Multi-language support and accessibility improvements.
8. Wearable and IoT integration, for example a smartwatch SOS.

References

1. Flutter documentation. https://docs.flutter.dev
2. Dart language documentation. https://dart.dev/guides
3. `provider` package. https://pub.dev/packages/provider
4. `sqflite` package. https://pub.dev/packages/sqflite
5. `sqflite_common_ffi` package. https://pub.dev/packages/sqflite_common_ffi
6. `geolocator` package. https://pub.dev/packages/geolocator
7. `url_launcher` package. https://pub.dev/packages/url_launcher
8. `vibration` package. https://pub.dev/packages/vibration
9. `audioplayers` package. https://pub.dev/packages/audioplayers
10. `intl` package. https://pub.dev/packages/intl
11. SQLite documentation. https://www.sqlite.org/docs.html
12. Android developers, permissions overview. https://developer.android.com/guide/topics/permissions/overview
13. Project document, Mini Project in Mobile Application Development, A D Patel Institute of Technology.